

Bloc 2 TP Dart Flutter

Contexte	2
Installation de Flutter	2
Exercices	7
Projet Flutter Dark Mode	36
Conclusion	41

Contexte

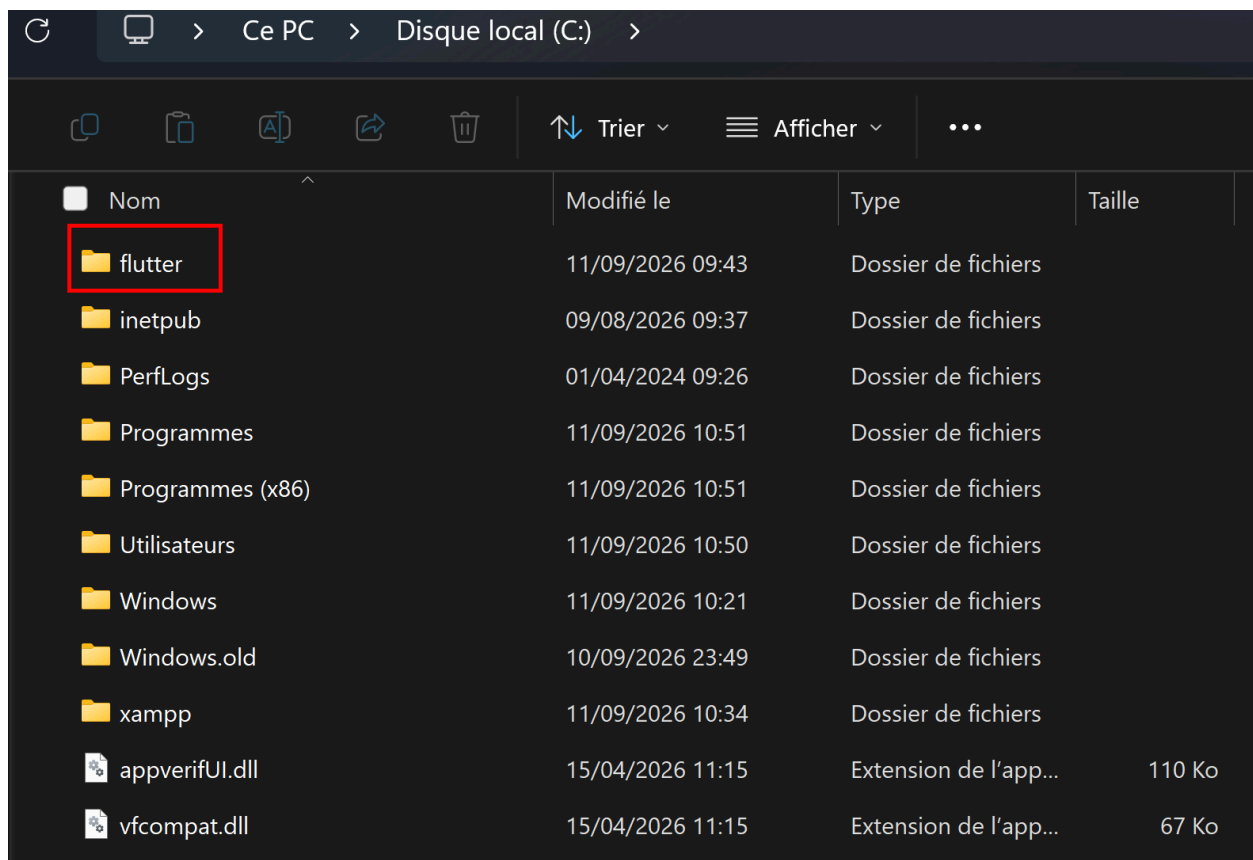
Ce TP va nous faire découvrir **Dart** et **Flutter**, il présentera leurs installation complètes, des exercices d'entraînement ainsi qu'un projet où nous pourrons y découvrir la gestion de thème (Light/Dark)

*Le projet **Dark Mode** est disponible sur **GitHub***

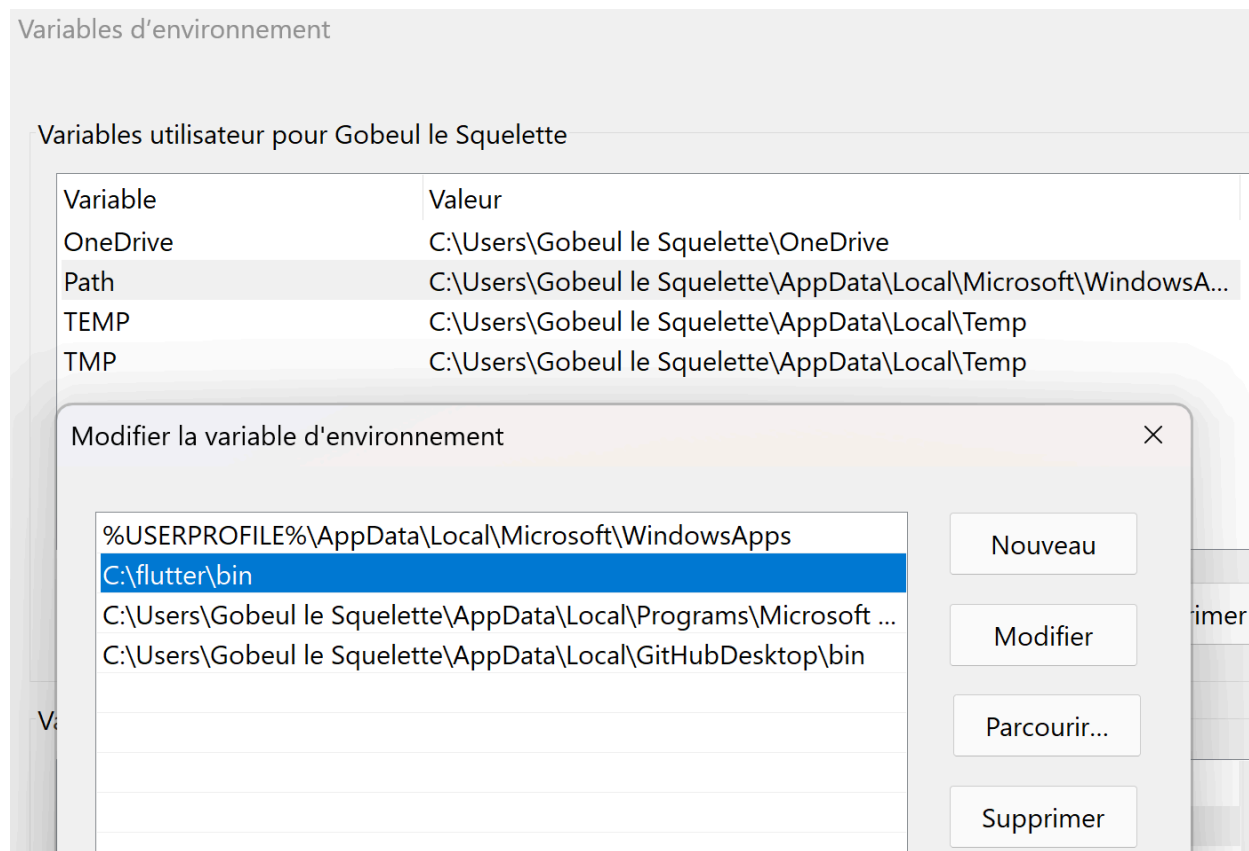
(https://github.com/RomAub/RomAub-bvl_S3_B2_TP_Dart_Flutter)

Installation de Flutter

Installation de **Flutter SDK** directement depuis le site officiel, puis je le place à la racine du disque **C** (idéalement) et je l'extrait ici :



Ensuite, pour pouvoir utiliser les commandes **Flutter** dans le **cmd**, je dois la définir dans les variables d'environnement :



Il est maintenant possible d'utiliser les commandes **Flutter** dans la **cmd**.

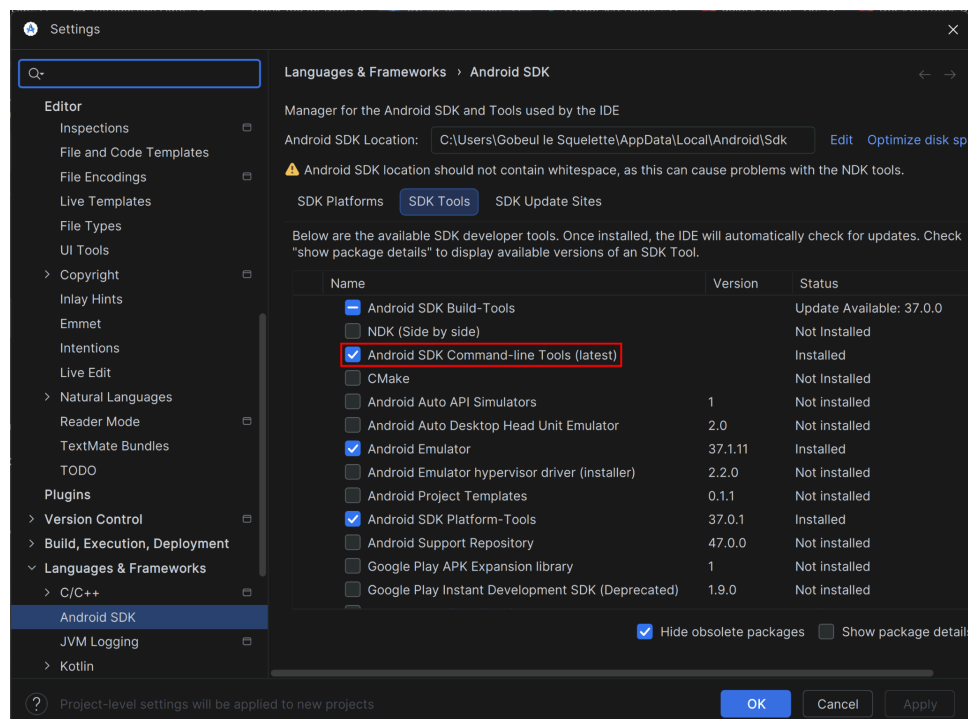
On vérifie maintenant que toutes les dépendance de **Flutter** sont bien présente avec “**flutter doctor**” :

```
Administrateur : Invite de commandes - flutter doctor
Microsoft Windows [version 10.0.26200.9445]
(c) Microsoft Corporation. Tous droits réservés.

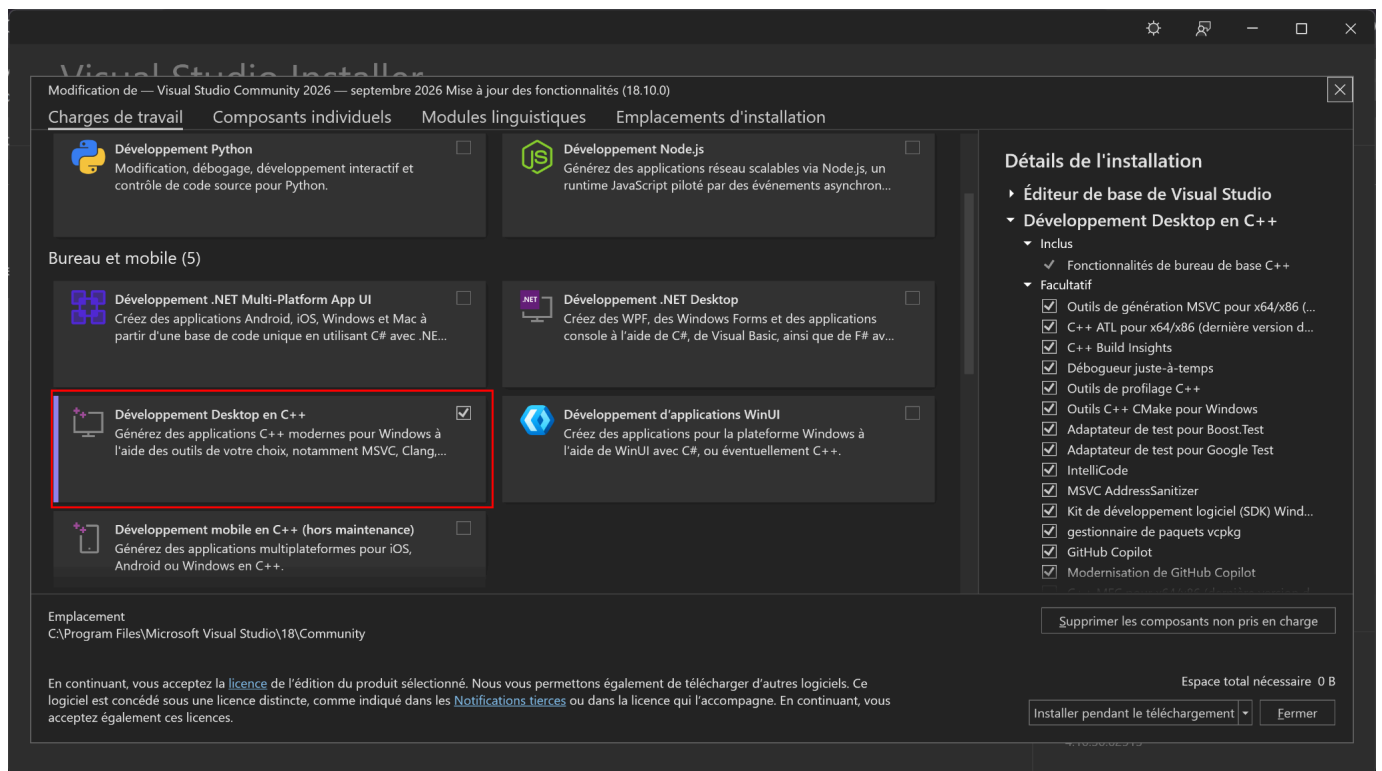
C:\Windows\System32>set HTTPS_PROXY=http://172.16.63.130:3128
C:\Windows\System32>set HTTP_PROXY=http://172.16.63.130:3128
C:\Windows\System32>flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.47.3, on Microsoft Windows [version 10.0.26200.9445], locale fr-FR)
[✓] Windows Version (Windows 11 or higher, 25H2, 2009)
[X] Android toolchain - develop for Android devices
    X cmdline-tools component is missing.
      Try installing or updating Android Studio.
      Alternatively, download the tools from https://developer.android.com/studio#command-line-tools-only and make sure
      to set the ANDROID_HOME environment variable.
      See https://developer.android.com/studio/command-line for more details.
[✓] Chrome - develop for the web
[X] Visual Studio - develop Windows apps
    X Visual Studio not installed; this is necessary to develop Windows apps.
      Download at https://visualstudio.microsoft.com/downloads/.
      Please install the "Desktop development with C++" workload, including all of its default components
[!] Proxy Configuration
    ! NO_PROXY is not set
[✓] Connected device (3 available)
[✓] Network resources

! Doctor found issues in 3 categories.
C:\Windows\System32>
```

L'erreur indique qu'il faut activé le composant “**cmdline-tools**” dans **SDK Tools** dans **Android Studio**, c'est un logiciel qui permet de simuler les téléphones.



Il faut ensuite installé **Visual Studio** avec l'extension “**Desktop development with C++**” :



On vérifie ensuite en exécutant “**flutter doctor**” :

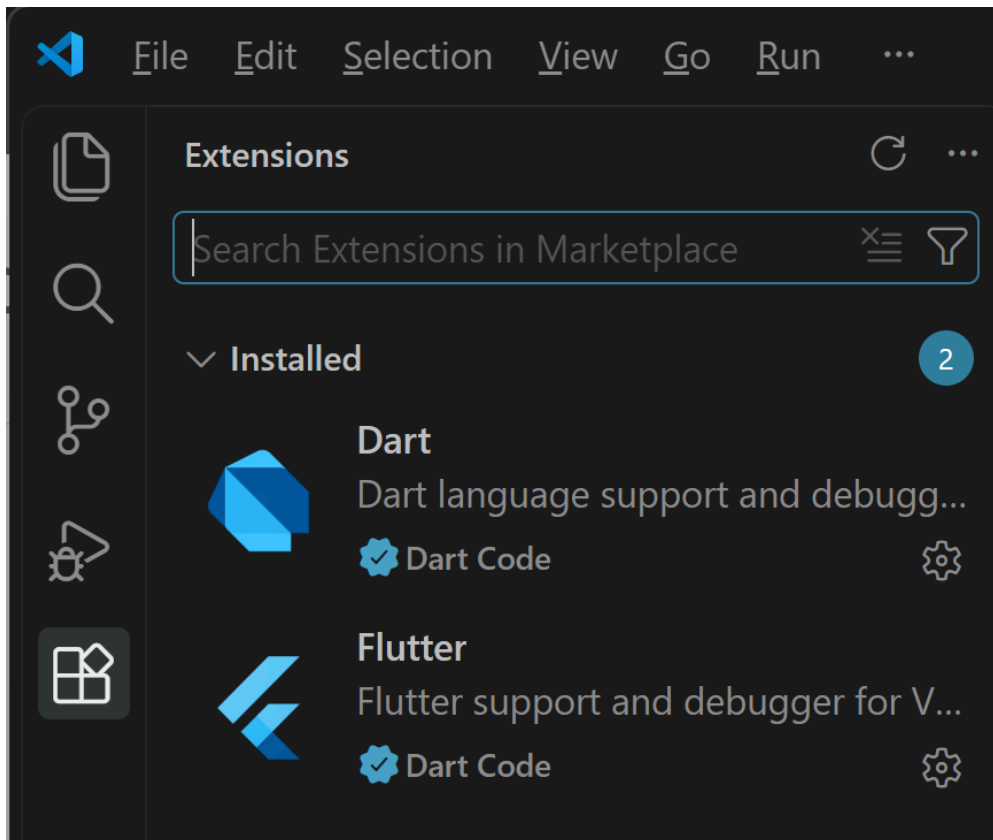
```
C:\Windows\System32>flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.47.3, on Microsoft Windows [version 10.0.26200.9445], locale fr-FR)
[✓] Windows Version (Windows 11 or higher, 25H2, 2009)
[✓] Android toolchain - develop for Android devices (Android SDK version 36.0.0)
[✓] Chrome - develop for the web
[✓] Visual Studio - develop Windows apps (Visual Studio Community 2026 18.10.0)
[!] Proxy Configuration
    ! NO_PROXY is not set
[✓] Connected device (3 available)
[✓] Network resources

! Doctor found issues in 1 category.

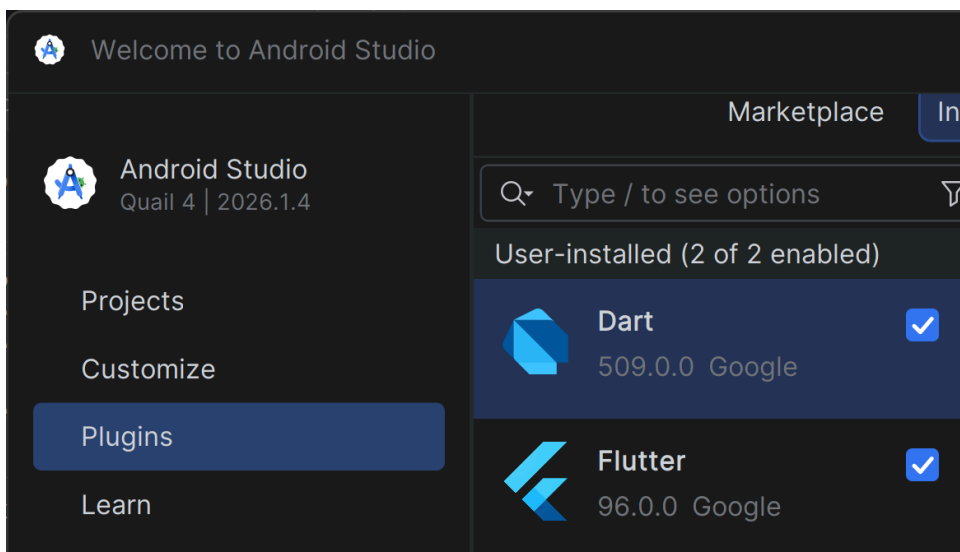
C:\Windows\System32>
```

Tout est bon (le **proxy** n'est pas grave).

On utilisera **VS Code**, donc il faut installer les extensions **Flutter** et **Dart**.



Pareil pour **Android Studio** :



Exercices

Q1 Programme qui demande à l'utilisateur d'entrer son nom et son âge. Puis qui affiche un message qui indique dans combien d'années vous aurez 100 ans :

```
import 'dart:io';

Run | Debug
void main() {
  stdout.write("Entrez votre nom : ");
  String? name = stdin.readLineSync();
  stdout.write("Entrez votre âge : ");
  int age = int.parse(stdin.readLineSync()!);
  int yearsTo100 = 100 - age;
  print("Bonjour $name, vous aurez 100 ans dans $yearsTo100 ans.");
}
```

```
Entrez votre nom : test
Entrez votre âge : 67
Bonjour test, vous aurez 100 ans dans 33 ans.
```

Q2 Demandez un numéro à l'utilisateur. Selon que le nombre est pair ou impair, imprimez un message approprié à l'utilisateur :

Tente une division entière par 2 pour différencier les nombres pair/impair.

```
import 'dart:io';

Run | Debug
void main() {
  stdout.write(object: "Entrez un nombre : ");
  int number = int.parse(source: stdin.readLineSync());
  if (number % 2 == 0) {
    print(object: "Le nombre $number est pair.");
  } else {
    print(object: "Le nombre $number est impair.");
  }
}
```

```
Entrez un nombre : 33
Le nombre 33 est impair.
```

Q3 Prenez une liste, dites par exemple celle-ci : “a = [1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]” et écrivez un programme qui imprime tous les éléments de la liste qui sont inférieurs à 5 :

```
void main() {
  List<int> a = <int>[1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89];
  for (int i in a) {
    if (i < 5) {
      print(object: i);
    }
  }
}
```

Cela renvoie 1, 1, 2, 3.

Q4 Créez un programme qui demande à l'utilisateur un nombre, puis imprime une liste de tous les diviseurs de ce nombre :

On tente de diviser le nombre par "i" qui augmentera jusqu'à la même valeur que le nombre entrée, si la division entière est bonne, elle est rajoutée à la liste, qui sera affichée ensuite.

```
import 'dart:io';  
Run | Debug  
void main() {  
  stdout.write(object: "Entrez un nombre : ");  
  int number = int.parse(source: stdin.readLineSync()!);  
  List<int> divisors = <int>[];  
  for (int i = 1; i <= number; i++) {  
    if (number % i == 0) {  
      divisors.add(value: i);  
    }  
  }  
  print(object: "Les diviseurs de $number sont : $divisors");  
}
```

Les diviseurs de 128 sont : [1, 2, 4, 8, 16, 32, 64, 128]

Q5 Prenons deux listes, par exemple : “a = [1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89] b = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13]” et écrivez un programme qui retourne une liste qui ne contient que les éléments qui sont communs entre eux (sans doublons). Assurez-vous que votre programme fonctionne sur deux listes de tailles différentes :

```
void main() {  
  List<int> a = <int>[1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89];  
  List<int> b = <int>[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13];  
  Set<int> common = <int>{};  
  for (int x in a) {  
    if (b.contains(element: x)) {  
      common.add(value: x);  
    }  
  }  
  print(object: "Éléments communs sans doublons : ${common.toList()}");  
}
```

Éléments communs sans doublons : [1, 2, 3, 5, 8, 13]

Q6 Demandez à l'utilisateur une chaîne et imprimez si cette chaîne est un palindrome ou non :

```
import 'dart:io';  
Run | Debug  
void main() {  
  stdout.write("Entrez une chaîne de caractères : ");  
  String input = stdin.readLineSync()!;  
  String reversed = input.split('').reversed.join('');  
  if (input == reversed) {  
    print("C'est un palindrome");  
  } else {  
    print("Ce n'est pas un palindrome");  
  }  
}
```

Entrez une chaîne de caractères : sos
C'est un palindrome

Q7 Disons qu'on vous donne une liste enregistrée dans une variable : “a = [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]” Écrivez un code Dart qui prend cette liste et en crée une nouvelle qui ne contient que les éléments pairs de cette liste :

```
void main() {  
  List<int> a = [1, 4, 9, 16, 25, 36, 49, 64, 81, 100];  
  List<int> aPair = [];  
  for (int i in a) {  
    if (i % 2 == 0) {  
      aPair.add(i);  
    }  
  }  
  print("Éléments pair de la liste : ${aPair.toList()}");  
}
```

Éléments pair de la liste : [4, 16, 36, 64, 100]

Q8 Faites un jeu Pierre-Papier-Ciseaux à deux joueurs contre l'ordinateur :

```
import 'dart:io';
import 'dart:math';

Run | Debug
void main() {
  List<String> options = ['pierre', 'papier', 'ciseaux'];
  print("Jeu Pierre-Papier-Ciseaux contre l'ordinateur !");
  stdout.write("Votre choix (pierre, papier, ciseaux) : ");
  String userChoice = stdin.readLineSync()!.toLowerCase();

  if (!options.contains(userChoice)) {
    print("Choix invalide.");
    return;
  }

  String computerChoice = options[Random().nextInt(options.length)];
  print("L'ordi a choisi : $computerChoice");

  if (userChoice == computerChoice) {
    print("Egalité");
  } else if ((userChoice == 'pierre' && computerChoice == 'ciseaux') ||
    (userChoice == 'papier' && computerChoice == 'pierre') ||
    (userChoice == 'ciseaux' && computerChoice == 'papier')) {
    print("Vous avez gagné !");
  } else {
    print("L'ordinateur a gagné !");
  }
}
```

```
Jeu Pierre-Papier-Ciseaux contre l'ordinateur !
Votre choix (pierre, papier, ciseaux) : papier
L'ordi a choisi : ciseaux
L'ordinateur a gagné !
```

Q9 Générez un nombre aléatoire entre 1 et 100. Demandez à l'utilisateur de deviner le nombre, puis dites-lui s'il a deviné trop bas, trop haut ou exactement juste :

```
import 'dart:io';
import 'dart:math';

Run | Debug
void main() {
  int target = Random().nextInt(max: 100) + 1;
  int guesses = 0;
  int? guess;

  print(object: "Devinez le nombre entre 1 et 100");
  while (guess != target) {
    stdout.write(object: "Votre choix : ");
    guess = int.parse(source: stdin.readLineSync()!);
    guesses++;

    if (guess < target) {
      print(object: "Trop bas");
    } else if (guess > target) {
      print(object: "Trop haut");
    } else {
      print(object: "Bravo, Trouvé en $guesses essais.");
    }
  }
}
```

```
Devinez le nombre entre 1 et 100
Votre choix : 47
Trop bas
Votre choix : 60
Trop haut
Votre choix : 55
Bravo, Trouvé en 3 essais.
```

Q10 Demandez à l'utilisateur un nombre et déterminez si le nombre est premier ou non :

```
import 'dart:io';

bool isPrime(int n) {
  if (n <= 1) return false;
  for (int i = 2; i <= n ~/ 2; i++) {
    if (n % i == 0) return false;
  }
  return true;
}

Run | Debug
void main() {
  stdout.write("Entrez un nombre : ");
  int num = int.parse(stdin.readLineSync());
  if (isPrime(num)) {
    print("$num est un nombre premier");
  } else {
    print("$num n'est pas un nombre premier");
  }
}
```

```
Entrez un nombre : 54
54 n'est pas un nombre premier
PS C:\Users\Gobeul le Squelette
Entrez un nombre : 63
63 n'est pas un nombre premier
PS C:\Users\Gobeul le Squelette
Entrez un nombre : 23
23 est un nombre premier
```

Q11 Écrire un programme qui prend une liste de nombres par exemple “a = [5, 10, 15, 20, 25]” et fait une nouvelle liste des seuls premiers et derniers éléments de la liste donnée. Pour vous entraîner, écrivez ce code dans une fonction :

first and *last* s’occupe de ça facilement.

```
List<int> firstAndLast(List<int> l) {  
    return [l.first, l.last];  
}  
  
Run | Debug  
void main() {  
    List<int> a = [5, 10, 15, 20, 25];  
    print("Premiers et derniers éléments : ${firstAndLast(a)}");  
}
```

```
Premiers et derniers éléments : [5, 25]
```

Q12 Écrivez un programme qui demande à l'utilisateur combien de nombres de Fibonnaci il doit générer, puis les génère. Profitez de cette occasion pour réfléchir à la façon dont vous pouvez utiliser les fonctions :

```
import 'dart:io';

List<int> generateFibonacci(int count) {
  List<int> fib = [1, 1];
  if (count <= 0) return [];
  if (count == 1) return [1];
  for (int i = 2; i < count; i++) {
    fib.add(fib[i - 1] + fib[i - 2]);
  }
  return fib;
}

Run | Debug
void main() {
  stdout.write("Combien de nombres de Fibonacci générer ? ");
  int n = int.parse(stdin.readLineSync());
  print("Séquence : ${generateFibonacci(n)}");
}
```

```
Combien de nombres de Fibonacci générer ? 14
Séquence : [1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377]
```

Q13 Écrivez un programme (fonction) qui prend une liste et renvoie une nouvelle liste qui contient tous les éléments de la première liste moins tous les doublons :

```
List<T> removeDuplicates<T>(List<T> list) {
  return list.toSet().toList();
}

Run | Debug
void main() {
  List<int> a = [1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 1, 2];
  print(object: "Sans doublons : ${removeDuplicates<int>(list: a)}");
}
```

Sans doublons : [1, 2, 3, 5, 8, 13, 21, 34, 55, 89]

Q14 Écrivez un programme (en utilisant des fonctions !) qui demande à l'utilisateur une longue chaîne contenant plusieurs mots. Renvoyez à l'utilisateur la même chaîne, sauf avec les mots dans l'ordre inverse :

```
import 'dart:io';

String reverseWords(String sentence) {
  return sentence.split(pattern: ' ').reversed.join(separator: ' ');
}

Run | Debug
void main() {
  stdout.write(object: "Entrez une phrase : ");
  String input = stdin.readLineSync();
  print(object: "Phrase inversée : ${reverseWords(sentence: input)}");
}
```

```
Entrez une phrase : castor magique de la buandrie
Phrase inversée : buandrie la de magique castor
```

Q15 Écrivez un générateur de mot de passe dans Dart. Soyez créatif avec la façon dont vous générez des mots de passe - les mots de passe forts contiennent un mélange de lettres minuscules, de lettres majuscules, de chiffres et de symboles. Les mots de passe doivent être aléatoires, générant un nouveau mot de passe chaque fois que l'utilisateur demande un nouveau mot de passe. Incluez votre code d'exécution dans une méthode principale :

```
import 'dart:io';
import 'dart:math';

Run | Debug
void main() {
  stdout.write("Quel niveau de mot de passe souhaitez-vous (faible/fort) ? ");
  String level = stdin.readLineSync()!.toLowerCase();

  if (level == 'faible') {
    print("Mot de passe faible : castor_bleu");
  } else {
    String chars =
      "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789@#\$%^&*()";
    Random rnd = Random();
    String password = String.fromCharCode(
      Iterable.generate(12, (_) => chars.codeUnitAt(rnd.nextInt(chars.length))),
    ); // String.fromCharCode
    print("Mot de passe fort généré : $password");
  }
}
```

```
Quel niveau de mot de passe souhaitez-vous (faible/fort) ? fort
Mot de passe fort généré : T10p&QGRKqNQ
```

Q16 Créez un programme qui jouera le jeu « des vaches et des taureaux » avec l'utilisateur. Le jeu fonctionne comme ceci :

- Générez aléatoirement un nombre à 4 chiffres. Demandez à l'utilisateur de deviner un nombre à 4 chiffres. Pour chaque chiffre que l'utilisateur a deviné correctement au bon endroit, il a une "vache". Pour chaque chiffre que l'utilisateur a deviné correctement au mauvais endroit est un "taureau".
- Chaque fois que l'utilisateur fait une supposition, dites-lui combien de "vaches" et de "taureaux" il a. Une fois que l'utilisateur a deviné le bon numéro, le jeu est terminé. Gardez une trace du nombre de suppositions que l'utilisateur fait tout au long du jeu et dites-le à l'utilisateur à la fin.

```
import 'dart:io';
import 'dart:math';

Run | Debug
void main() {
  String secret = (Random().nextInt(9000) + 1000).toString();
  int guesses = 0;
  print("--- Jeu des vaches et taureaux ---");

  while (true) {
    stdout.write("Devinez un nombre à 4 chiffres : ");
    String guess = stdin.readLineSync()!;
    guesses++;

    if (guess == secret) {
      print("Bravo, vous avez trouvé le nombre en $guesses essais.");
      break;
    }

    int cows = 0;
    int bulls = 0;

    for (int i = 0; i < 4; i++) {
      if (guess[i] == secret[i]) {
        cows++;
      } else if (secret.contains(guess[i])) {
        bulls++;
      }
    }
    print("Résultat : $cows vaches, $bulls taureaux.");
  }
}
```

Résultat :

```
--- Jeu des vaches et taureaux ---  
Devinez un nombre à 4 chiffres : 5155  
Résultat : 1 vaches, 2 taureaux.  
Devinez un nombre à 4 chiffres : 1555  
Résultat : 0 vaches, 3 taureaux.  
Devinez un nombre à 4 chiffres : 1000  
Résultat : 0 vaches, 0 taureaux.  
Devinez un nombre à 4 chiffres : 5000  
Résultat : 1 vaches, 0 taureaux.  
Devinez un nombre à 4 chiffres : 5100  
Résultat : 1 vaches, 0 taureaux.  
Devinez un nombre à 4 chiffres : 5147  
Résultat : 3 vaches, 0 taureaux.  
Devinez un nombre à 4 chiffres : 5140  
Résultat : 2 vaches, 0 taureaux.  
Devinez un nombre à 4 chiffres : 5197  
Résultat : 2 vaches, 1 taureaux.  
Devinez un nombre à 4 chiffres : 5947  
Bravo, vous avez trouvé le nombre en 9 essais.
```

Q17 C'est l'heure des faux graphismes ! Disons que nous voulons dessiner des plateaux de jeu qui ressemblent à ceci :

```
--- --- ---
| | | |
--- --- ---
| | | |
--- --- ---
| | | |
--- --- ---
```

Celui-ci est 3x3 (comme dans tic tac toe). Demandez à l'utilisateur quelle taille de plateau de jeu il souhaite dessiner et dessinez-le pour lui à l'écran à l'aide de l'instruction d'impression de Dart.

```
void printBoard(int size) {
  for (int i = 0; i < size; i++) {
    print(object: " ---" * size);
    print(object: "|   " * (size + 1));
  }
  print(object: " ---" * size);
}
```

Run | Debug

```
void main() {
  printBoard(size: 3); // 3x3
}
```

```
--- --- ---
| | | |
--- --- ---
| | | |
--- --- ---
| | | |
--- --- ---
```

Si on met “7” par exemple, ça donnera ceci :



Q18 Étant donné une liste 3 par 3 de listes représentant un plateau de jeu Tic Tac Toe, indiquez si quelqu'un a gagné et indiquez quel joueur a gagné, le cas échéant :

```
int checkWinner(List<List<int>> game) {
    // lignes
    for (int i = 0; i < 3; i++) {
        if (game[i][0] != 0 && game[i][0] == game[i][1] && game[i][1] == game[i][2]) {
            return game[i][0];
        }
    }
    // colonnes
    for (int i = 0; i < 3; i++) {
        if (game[0][i] != 0 && game[0][i] == game[1][i] && game[1][i] == game[2][i]) {
            return game[0][i];
        }
    }
    // diagonale
    if (game[0][0] != 0 && game[0][0] == game[1][1] && game[1][1] == game[2][2]) {
        return game[0][0];
    }
    if (game[0][2] != 0 && game[0][2] == game[1][1] && game[1][1] == game[2][0]) {
        return game[0][2];
    }
    return 0; // Pas de gagnant
}

Run | Debug
void main() {
    List<List<int>> game = <List<int>>[
        <int>[1, 2, 0],
        <int>[2, 1, 0],
        <int>[2, 1, 1]
    ];
    int winner = checkWinner(game: game);
    print(object: "Le gagnant est le joueur : $winner");
}
```

Le gagnant est le joueur : 1

Q19 La prochaine étape logique consiste à traiter la gestion des entrées de l'utilisateur. Lorsqu'un joueur (disons le joueur 1, qui est X) veut placer un X sur l'écran, il ne peut pas simplement cliquer sur un terminal. Vous allez donc approximer ce clic simplement en demandant à l'utilisateur une coordonnée de l'endroit où il souhaite placer sa pièce :

```
import 'dart:io';

void playerMove(List<List<int>> board, int playerSymbol) {
  print("Joueur $playerSymbol, à vous de jouer.");

  stdout.write("Entrez le numéro de ligne (0, 1 ou 2) : ");
  int row = int.parse(stdin.readLineSync()!);

  stdout.write("Entrez le numéro de colonne (0, 1 ou 2) : ");
  int col = int.parse(stdin.readLineSync()!);

  if (row >= 0 && row < 3 && col >= 0 && col < 3) {
    if (board[row][col] == 0) {
      board[row][col] = playerSymbol;
    } else {
      print("Case déjà occupée ! Choisissez une autre case.");
      playerMove(board, playerSymbol); // retry
    }
  } else {
    print("Coordonnées invalides. Entrez des chiffres entre 0 et 2.");
    playerMove(board, playerSymbol);
  }
}

Run | Debug
void main() {
  List<List<int>> board = [
    [0, 0, 0],
    [0, 0, 0],
    [0, 0, 0],
  ];
  playerMove(board, 1);
  print("Plateau mis à jour : $board");
}
```

```
Joueur 1, à vous de jouer.
Entrez le numéro de ligne (0, 1 ou 2) : 2
Entrez le numéro de colonne (0, 1 ou 2) : 1
Plateau mis à jour : [[0, 0, 0], [0, 0, 0], [0, 1, 0]]
```

Q20 Dans les 3 exercices précédents, nous avons construit quelques composants nécessaires pour construire un jeu de Tic Tac Toe dans Dart :

- Dessinez le plateau de jeu Tic Tac Toe
- Vérifier si un plateau de jeu a un gagnant
- Gérer un mouvement de joueur à partir de l'entrée de l'utilisateur

L'étape suivante consiste à assembler ces trois composants pour créer un jeu Tic Tac Toe à deux joueurs !

Votre défi dans cet exercice est d'utiliser toutes les fonctions de ces exercices précédents dans le même programme pour créer un jeu à deux joueurs auquel vous pouvez jouer avec un ami. Il y a beaucoup de choix que vous devez faire lorsque vous terminerez cet exercice, vous pouvez donc aller aussi loin ou aussi peu que vous le souhaitez.

Voici quelques éléments à garder à l'esprit :

- Vous devez garder une trace de qui a gagné - s'il y a un gagnant, affichez un message de félicitations à l'écran.
- S'il ne reste plus de coup, ne demandez pas le coup du joueur suivant !

```
import 'dart:io';

// Affichage du plateau en mode texte
void printBoard(List<List<int>> board) {
  for (var row in board) {
    String line = row.map((cell) {
      if (cell == 1) return 'X';
      if (cell == 2) return 'O';
      return ' ';
    }).join(' | ');
    print(line);
    print('-----');
  }
}

// Vérif gagnant exo18
int checkWinner(List<List<int>> game) {
  for (int i = 0; i < 3; i++) {
    if (game[i][0] != 0 && game[i][0] == game[i][1] && game[i][1] == game[i][2]) return game[i][0];
    if (game[0][i] != 0 && game[0][i] == game[1][i] && game[1][i] == game[2][i]) return game[0][i];
  }
  if (game[0][0] != 0 && game[0][0] == game[1][1] && game[1][1] == game[2][2]) return game[0][0];
  if (game[0][2] != 0 && game[0][2] == game[1][1] && game[1][1] == game[2][0]) return game[0][2];
  return 0;
}
```

```

void main() {
    List<List<int>> board = <List<int>>[
        <int>[0, 0, 0],
        <int>[0, 0, 0],
        <int>[0, 0, 0]
    ];

    int turns = 0;
    int currentPlayer = 1;

    print(object: "--- Jeu du Tic-Tac-Toe (Joueur 1: X, Joueur 2: O) ---");

    while (turns < 9) {
        printBoard(board: board);
        print(object: "\nTour du Joueur $currentPlayer (${currentPlayer == 1 ? 'X' : 'O'})");

        stdout.write(object: "Ligne (0-2) : ");
        int row = int.parse(source: stdin.readLineSync());
        stdout.write(object: "Colonne (0-2) : ");
        int col = int.parse(source: stdin.readLineSync());

        if (board[row][col] == 0) {
            board[row][col] = currentPlayer;
            turns++;

            int winner = checkWinner(game: board);
            if (winner != 0) {
                printBoard(board: board);
                print(object: "\n Félicitations ! Le Joueur $winner (${winner == 1 ? 'X' : 'O'}) a gagné !");
                return;
            }
        }
    }
}

```

```

        // Alternance des joueurs
        currentPlayer = (currentPlayer == 1) ? 2 : 1;
    } else {
        print(object: "Cette case est déjà prise, réessayez.");
    }
}

printBoard(board: board);
print(object: "\nMatch nul ! Plus de coups disponibles.");
}

```

Aperçu du jeu :

Tour du Joueur 2 (O)

Ligne (0-2) : 1

Colonne (0-2) : 0

X | X |

O | O |

| |

Tour du Joueur 1 (X)

Ligne (0-2) : 0

Colonne (0-2) : 2

X | X | X

O | O |

| |

Félicitations ! Le Joueur 1 (X) a gagné !

Q21 Vous, l'utilisateur, aurez dans votre tête un nombre compris entre 0 et 100. Le programme devinera un nombre, et vous, l'utilisateur, direz s'il est trop haut, trop bas ou votre nombre :

```
import 'dart:io';

Run | Debug
void main() {
  print(object: "Pensez à un nombre entre 0 et 100");
  int min = 0;
  int max = 100;
  int guesses = 0;

  while (true) {
    int guess = (min + max) ~/ 2;
    guesses++;
    stdout.write(object: "Est-ce que c'est $guess ? (trop haut / trop bas / juste) : ");
    String feedback = stdin.readLineSync()!.toLowerCase();

    if (feedback == 'juste') {
      print(object: "Trouvé en $guesses essais");
      break;
    } else if (feedback == 'trop haut') {
      max = guess - 1;
    } else if (feedback == 'trop bas') {
      min = guess + 1;
    }
  }
}
```

```
Pensez à un nombre entre 0 et 100
Est-ce que c'est 50 ? (trop haut / trop bas / juste) : trop bas
Est-ce que c'est 75 ? (trop haut / trop bas / juste) : trop haut
Est-ce que c'est 62 ? (trop haut / trop bas / juste) : trp bas
Est-ce que c'est 62 ? (trop haut / trop bas / juste) : trop bas
Est-ce que c'est 68 ? (trop haut / trop bas / juste) : trop haut
Est-ce que c'est 65 ? (trop haut / trop bas / juste) : trop bas
Est-ce que c'est 66 ? (trop haut / trop bas / juste) : trop bas
Est-ce que c'est 67 ? (trop haut / trop bas / juste) : juste
Trouvé en 8 essais
```

Q22 Implémentez une fonction qui prend en entrée trois variables et renvoie la plus grande des trois. Faites cela sans utiliser la fonction Dart `max()` ! :

```
int maxOfThree(int a, int b, int c) {  
  if (a >= b && a >= c) {  
    return a;  
  } else if (b >= a && b >= c) {  
    return b;  
  } else {  
    return c;  
  }  
}
```

Run | Debug

```
void main() {  
  int resultat = maxOfThree(10, 45, 22);  
  print("Le plus grand nombre est : $resultat");  
}
```

Le plus grand nombre est : 45

Q23 Dans cet exercice, la tâche consiste à écrire une fonction qui sélectionne un mot au hasard dans une liste de mots du dictionnaire SOWPODS :

```
import 'dart:io';
import 'dart:math';
💡
Run | Debug
void main() async {
  List<String> words = await File(path: 'sowpods.txt').readAsLines();
  String randomWord = words[Random().nextInt(max: words.length)];
  print(object: "Mot sélectionné au hasard : $randomWord");
}
```

Mot sélectionné au hasard : CYSTINURIAS

Q24 Dans le jeu du Pendu, un mot indice est donné par le programme que le joueur doit deviner, lettre par lettre. Le joueur devine une lettre à la fois jusqu'à ce que le mot entier ait été deviné. (Dans le jeu réel, le joueur ne peut deviner que 6 lettres incorrectes avant de perdre) :

```
import 'dart:io';
Run | Debug
void main() {
  String word = "PHYTOPHAGE";
  List<String> guessedLetters = [];

  print("--- Pendu : Partie 1 ---");
  while (true) {
    stdout.write("Devinez une lettre : ");
    String letter = stdin.readLineSync()!.toUpperCase();

    if (guessedLetters.contains(letter)) {
      print("Vous avez déjà essayé cette lettre");
      continue;
    }
    guessedLetters.add(letter);

    String display = "";
    bool allGuessed = true;

    for (int i = 0; i < word.length; i++) {
      if (guessedLetters.contains(word[i])) {
        display += word[i] + " ";
      } else {
        display += "_ ";
        allGuessed = false;
      }
    }

    print("Mot actuel : $display");

    if (allGuessed) {
      print("Félicitations, vous avez deviné le mot entier");
      break;
    }
  }
}
```

Résultat :

```
--- Pendu : Partie 1 ---  
Devinez une lettre : P  
Mot actuel : P _ _ _ P _ _ _  
Devinez une lettre : H  
Mot actuel : P H _ _ P H _ _  
Devinez une lettre : O  
Mot actuel : P H _ O P H _ _  
Devinez une lettre : Y  
Mot actuel : P H Y _ O P H _ _  
Devinez une lettre : T  
Mot actuel : P H Y T O P H _ _  
Devinez une lettre : A  
Mot actuel : P H Y T O P H A _ _  
Devinez une lettre : G  
Mot actuel : P H Y T O P H A G _  
Devinez une lettre : E  
Mot actuel : P H Y T O P H A G E  
Félicitations, vous avez deviné le mot entier
```

Q25 Dans cet exercice, nous terminerons la construction de Hangman. Dans le jeu du Pendu, le joueur n'a que 6 suppositions incorrectes (tête, corps, 2 jambes et 2 bras) avant de perdre la partie :

Code 1/2 :

```
import 'dart:io';
import 'dart:math';

Run | Debug
void main() async {
  // exo23
  File file = File('sowpods.txt');
  if (!await file.exists()) {
    print("Erreur : Le fichier sowpods.txt est introuvable.");
    return;
  }

  List<String> words = await file.readAsLines();
  String word = words[Random().nextInt(words.length)].toUpperCase();

  // 24/25
  List<String> guessedLetters = [];
  int attemptsLeft = 6;

  print("--- Jeu du Pendu Complet ---");

  // 25
  while (attemptsLeft > 0) {
    stdout.write("\nIl vous reste $attemptsLeft erreurs permises, Devinez une lettre : ");
    String letter = stdin.readLineSync()!.toUpperCase();

    if (guessedLetters.contains(letter)) {
      print("Lettre déjà devinée, essayez à nouveau.");
      continue;
    }
    guessedLetters.add(letter);
```

Code 2/2 :

```
if (!word.contains(other: letter)) {
    attemptsLeft--;
    print(object: "Mauvaise supposition !");
}

String display = "";
bool allGuessed = true;

for (int i = 0; i < word.length; i++) {
    if (guessedLetters.contains(element: word[i])) {
        display += word[i] + " ";
    } else {
        display += "_ ";
        allGuessed = false;
    }
}

print(object: "Mot : $display");

if (allGuessed) {
    print(object: "Victoire, Vous avez deviné le mot entier");
    return;
}

print(object: "Game Over, Le mot secret était : $word");
}
```

```
Il vous reste 5 erreurs permises, Devinez une lettre : V
Mauvaise supposition !
Mot : R _ _ A R _ _

Il vous reste 4 erreurs permises, Devinez une lettre : D
Mot : R _ _ A R D _

Il vous reste 4 erreurs permises, Devinez une lettre : C
Mauvaise supposition !
Mot : R _ _ A R D _

Il vous reste 3 erreurs permises, Devinez une lettre : L
Mauvaise supposition !
Mot : R _ _ A R D _

Il vous reste 2 erreurs permises, Devinez une lettre : U
Mauvaise supposition !
Mot : R _ _ A R D _

Il vous reste 1 erreurs permises, Devinez une lettre : R
Lettre déjà devinée, essayez à nouveau.

Il vous reste 1 erreurs permises, Devinez une lettre : T
Mauvaise supposition !
Mot : R _ _ A R D _
Game Over, Le mot secret était : RIZARDS
```

Q26 Pour cet exercice, nous garderons une trace de la date d'anniversaire de nos amis et serons en mesure de trouver ces informations en fonction de leur nom. Créez un dictionnaire (dans votre fichier) des noms et des anniversaires. Lorsque vous exécutez votre programme, il doit demander à l'utilisateur d'entrer un nom et lui renvoyer l'anniversaire de cette personne :

```
import 'dart:io';

Run | Debug
void main() {
  Map<String, String> birthdays = <String, String>{
    "Albert Einstein": "14/03/1879",
    "Benjamin Franklin": "17/01/1706",
    "Ada Lovelace": "10/12/1815",
  };

  print(
    object: "Bienvenue dans le dictionnaire d' anniversaire. Nous connaissons les dates d' anniversaire de:",
  );
  birthdays.keys.forEach(action: (String name) {
    print(object: name);
  });

  print(object: "De qui voulez vous connaitre la date d' anniversaire ?");
  String name = stdin.readLineSync();

  if (birthdays.containsKey(key: name)) {
    print(object: "$name est né le ${birthdays[name]}");
  } else {
    print(object: "Désolé, ce nom ne figure pas dans le dictionnaire.");
  }
}
```

```
Bienvenue dans le dictionnaire d' anniversaire. Nous connaissons les dates d' anniversaire de:
Albert Einstein
Benjamin Franklin
Ada Lovelace
De qui voulez vous connaitre la date d' anniversaire ?
Ada Lovelace
Ada Lovelace est né le 10/12/1815.
```

Projet Flutter Dark Mode

Le code sera disponible sur GitHub

(https://github.com/RomAub/RomAub-bvl_S3_B2_TP_Dart_Flutter)

Le but de ce projet est de créer un **Dark Mode** de l'application fournie en **PJ** dans le **TP**.

Création du projet “**tp_darkmode**” dans **VS Code** :

```
PS C:\Users\Gobeul le Squelette\Documents\Dark Flutter> flutter create tp_darkmode
>> cd tp_darkmode
Creating project tp_darkmode...
Resolving dependencies in `tp_darkmode`...
Downloading packages...
Got dependencies in `tp_darkmode`.
Wrote 131 files.

All done!
You can find general documentation for Flutter at: https://docs.flutter.dev/
Detailed API documentation is available at: https://api.flutter.dev/
If you prefer video documentation, consider: https://www.youtube.com/c/flutterdev

In order to run your application, type:

$ cd tp_darkmode
$ flutter run

Your application code is in tp_darkmode\lib\main.dart.
```

Installation du package “**adaptive_theme**” dans “**pubspec.yaml**” (sur **Windows**, il nécessite d'activer le mode développeur dans les paramètres pour installer certains packages) :

```
dependencies:
  flutter:
    sdk: flutter

# The following adds the Cupertino
# Use with the Cupertino
cupertino_icons: ^1.0.8
adaptive_theme: ^1.1.0
```

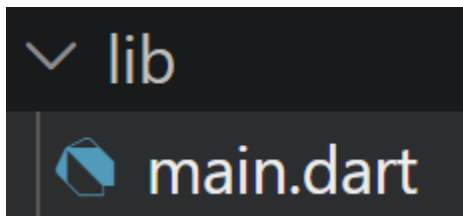
Puis on fait “**flutter pub get**” pour l'installer.

Cependant cela ne fonctionnera pas, car cette ancienne version ne possède pas le “**null safety**”, une règle de code stricte devenue obligatoire avec le **SDK Dart 3.13.3**, On va donc essayer d’installer la version **2.2.0** de **adaptive_theme**.

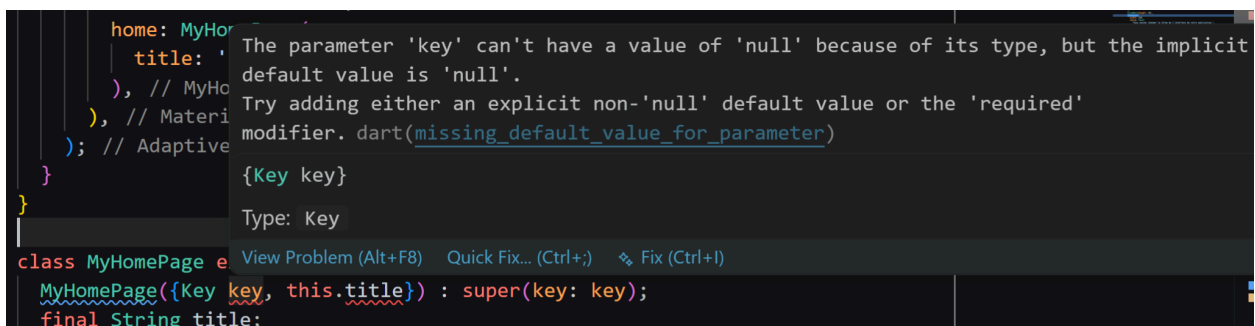
On refais donc “**flutter pub get**” :

```
PS C:\Users\Gobeul le Squelette\Documents\Dark Flutter\tp_darkmode> flutter pub get
Resolving dependencies...
Downloading packages...
  adaptive_theme 2.3.1 (3.8.0 available)
  material_color_utilities 0.13.0 (0.13.1 available)
  meta 1.18.3 (1.19.0 available)
  test_api 0.7.12 (0.7.14 available)
  vector_math 2.4.0 (2.4.2 available)
Got dependencies!
5 packages have newer versions incompatible with dependency constraints.
Try `flutter pub outdated` for more information.
```

Ensuite, on import le “**main.dart**” fournie en **PJ** et on le remplace (**lib/main.dart**) :



Il y a cependant des soucis dans le code actuelle, justement à cause du “**null safety**” des dernière version de **Dart** :



On doit donc remplacer “**MyHomePage({Key key, this.title}) : super(key: key);**” par “**const MyHomePage({super.key, required this.title});**”.

Le même soucis peut-être vue ici :

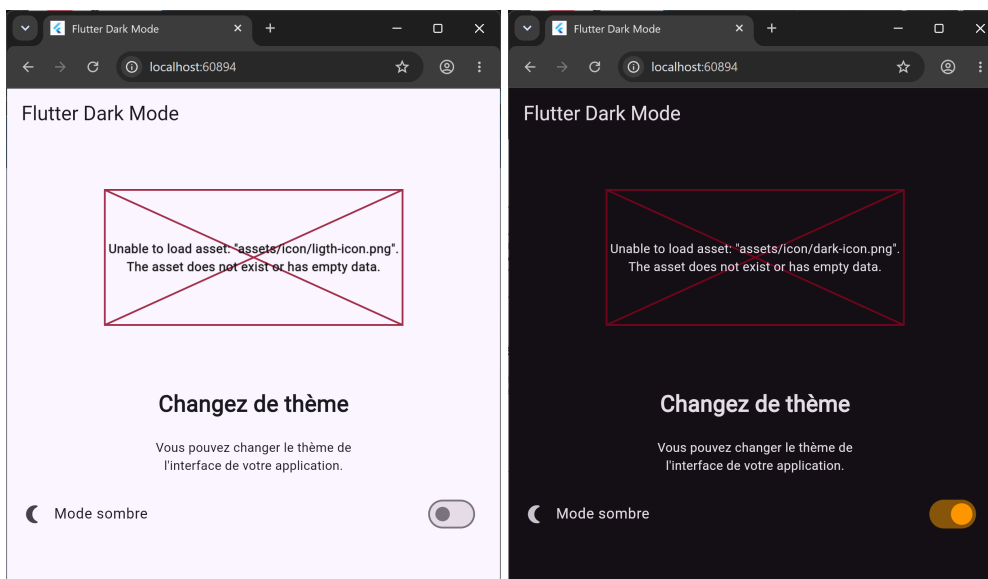
```
String iconAdress;
```

On doit simplement mettre “**String?**” pour l'autoriser à être “**null**”

```
child: iconAdress != null ? Image.asset(name: iconAdress) : Container(),
```

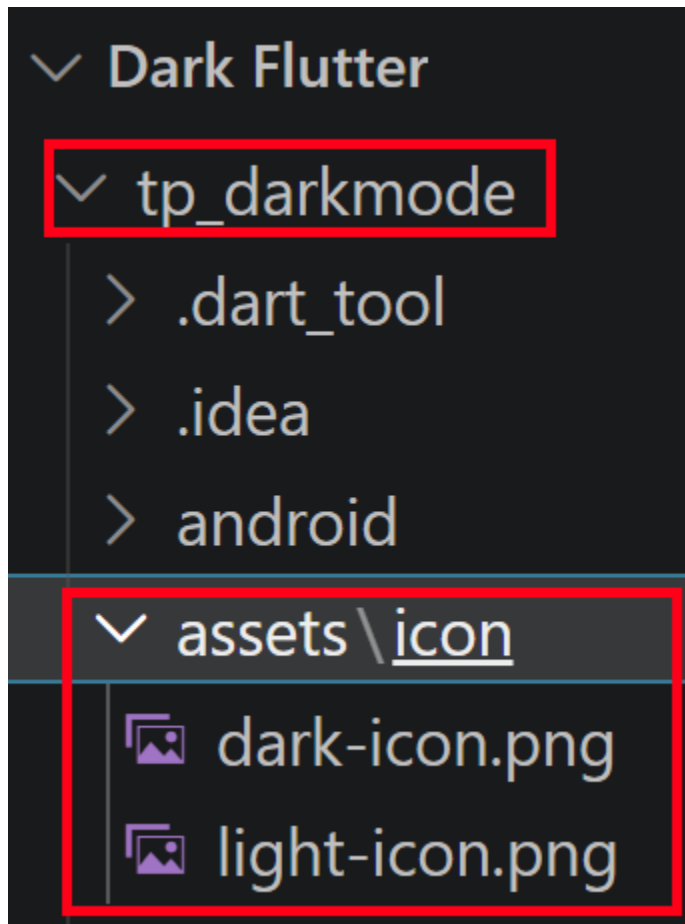
Il suffit de mettre “**iconAdress!**” avec un “**!**” à la fin.

On va pouvoir exécuter “**flutter run**” pour visualiser le rendu actuelle de l’application (sur **Google** ici) :



(Il n’y a pas d’icons pour l’instant).

Ajout des 2 icônes :



Aperçu des icons :



On doit légèrement modifier le code, l'ancien code avait inscrit "**assets/icon/**", ce qui voulait dire que le programme allé regardé dans "**assets/assets/icon**" :

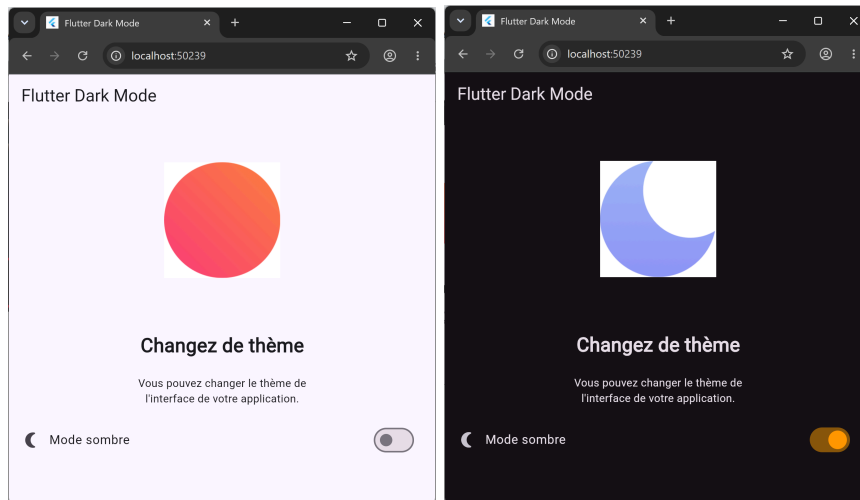
```
if (value == true) {  
  AdaptiveTheme.of(context: context).setDark();  
  iconAdress = 'icon/dark-icon.png';  
} else {  
  AdaptiveTheme.of(context: context).setLight();  
  iconAdress = 'icon/light-icon.png';  
}
```

("light" avais aussi une coquille, c'était écris "**ligth**")

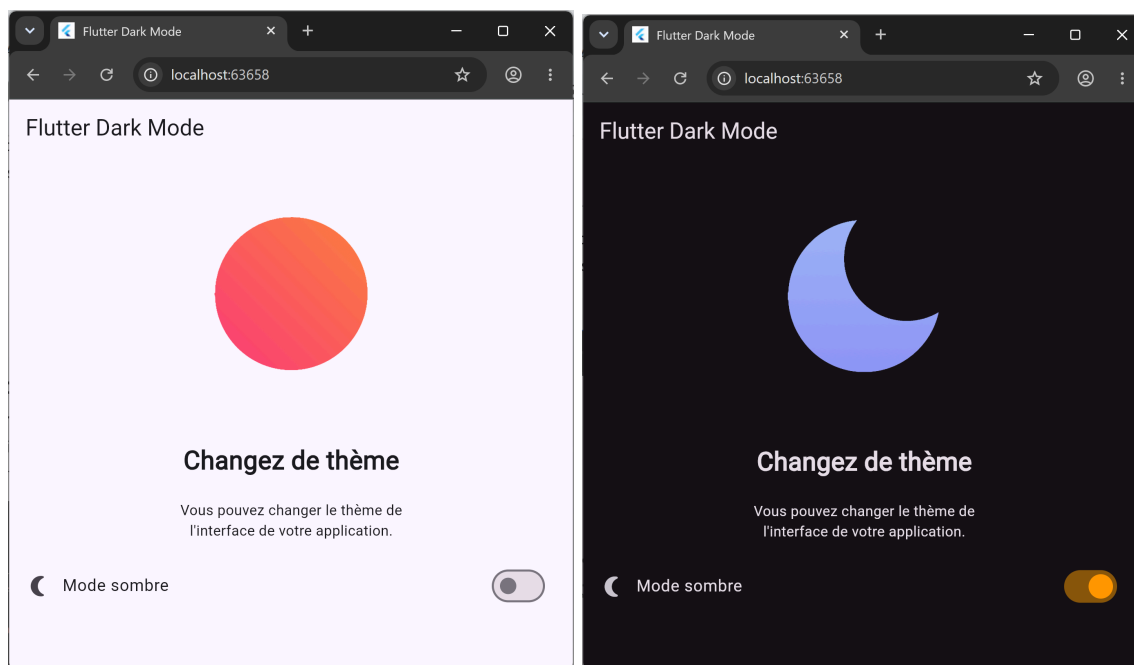
Il faut aussi modifier "**pubspec.yaml**" pour lui indiquer qu'il faut qu'il regarde dans le dossier **/icon/**.

```
flutter:  
  
# The following line ensures  
# included with your applica  
# the material Icons class.  
uses-material-design: true  
  
# To add assets to your appl  
assets:  
  - assets/icon/
```

Les icônes sont belle et bien affiché :



Il y a un petit soucis, les icônes ne sont pas transparente, ce qui donne cette apparence étrange, je vais supprimé le fond blanc sur “**Paint.NET**”.



Conclusion

Ce TP nous a introduit l’installation de **Dart** et **Flutter**, avec une grande série d’exercices ensuite ainsi qu’un projet pour nous apprendre à l'utiliser.